

HOSTEL MANAGEMENT SYSTEM

Project Report

1. Project Title

Hostel Management System Using Spring Boot and MongoDB

2. Objective

The Hostel Management System is a backend-based web application developed using Spring Boot and MongoDB. The system is designed to manage hostel-related operations such as student records, room allocation, hostel information, staff details, fee management, visitor records, departments, and complaints.

The application provides REST APIs that can be tested using Postman and integrated with any frontend application in the future.

3. Technologies Used

Technology	Version
Java	17
Spring Boot	3.x
MongoDB	6.x / Latest
Maven	3.9+
Spring Data MongoDB	Latest
IntelliJ IDEA	2024+
Postman	Latest
Swagger OpenAPI	Latest
Git & GitHub	Latest

4. System Architecture

The application follows a layered architecture:

Controller Layer



Service Layer



Repository Layer



MongoDB Database

Components

Controller Layer

Handles HTTP requests and responses.

Service Layer

Contains business logic.

Repository Layer

Performs database operations using Spring Data MongoDB.

Model Layer

Contains entity classes.

Database

MongoDB stores hostel management records.

5. Modules Implemented

The project consists of the following modules:

Student Management

- Add Student
- View Students

- Update Student
- Delete Student

Hostel Management

- Add Hostel
- View Hostel Details
- Update Hostel
- Delete Hostel

Room Management

- Manage Room Information
- Room Availability Tracking

Staff Management

- Add Staff
- Update Staff
- Delete Staff

Fee Management

- Fee Collection Records
- Payment Tracking

Department Management

- Manage Departments

Visitor Management

- Visitor Entry Records

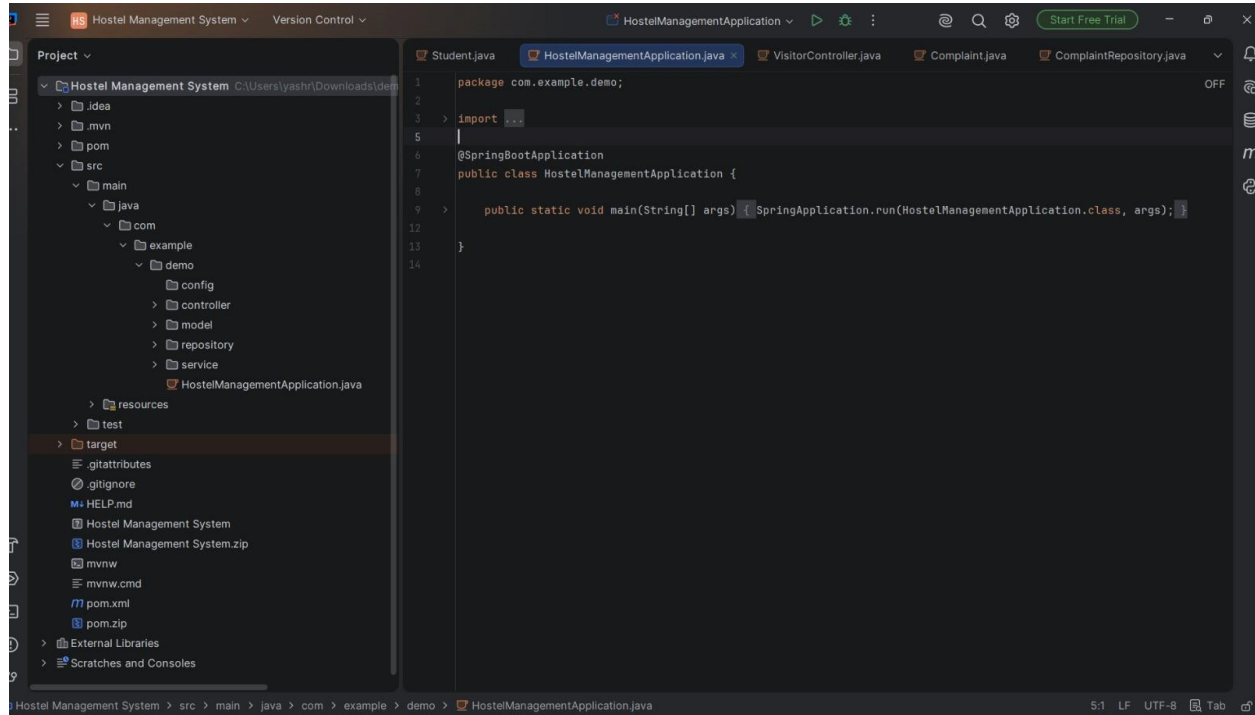
Complaint Management

- Register Complaints
- Track Complaints

6. Project Structure

Screenshot 1: Complete Project Structure

Explanation



This screenshot demonstrates the complete Spring Boot project architecture.

The project contains:

Controller Package

- ComplaintController
- DepartmentController
- FeeController
- HostelController
- RoomController
- StaffController
- StudentController
- VisitorController

These classes expose REST APIs.

Model Package

- Complaint
- Department
- Fee
- Hostel
- Room
- Staff
- Student
- Visitor

These classes represent MongoDB documents.

Repository Package

- ComplaintRepository
- DepartmentRepository
- FeeRepository
- HostelRepository
- RoomRepository
- StaffRepository
- StudentRepository
- VisitorRepository

Repositories provide database CRUD operations.

Service Package

- ComplaintService
- DepartmentService
- FeeService

- HostelService
- RoomService
- StaffService
- StudentService
- VisitorService

Services implement business logic.

Resources

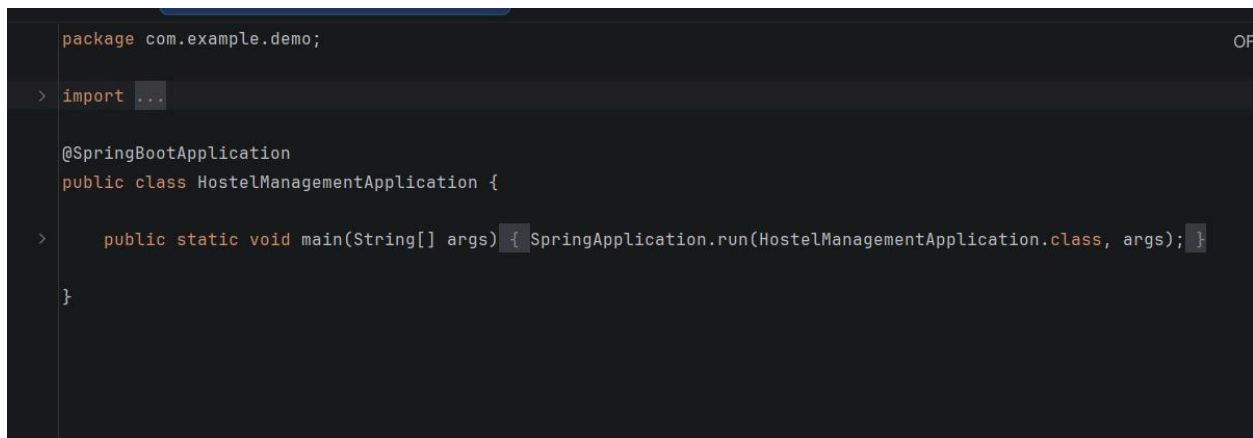
Contains:

application.properties

which stores MongoDB configuration.

7. Main Application Class

Screenshot 2: HostelManagementApplication.java



```
package com.example.demo;

import ...

@SpringBootApplication
public class HostelManagementApplication {

    public static void main(String[] args) { SpringApplication.run(HostelManagementApplication.class, args); }

}
```

Explanation

The application starts from the following class:

@SpringBootApplication

public class HostelManagementApplication {

```
public static void main(String[] args) {  
    SpringApplication.run(  
        HostelManagementApplication.class,  
        args  
    );  
}  
}
```

Purpose

- Bootstraps Spring Boot application.
 - Performs component scanning.
 - Initializes embedded server.
 - Connects application with MongoDB.
-

8. MongoDB Database Configuration

Configuration

Stored inside:

src/main/resources/application.properties

Example:

spring.data.mongodb.uri=mongodb://localhost:27017/hosteldb

spring.data.mongodb.database=hosteldb

server.port=8080

Purpose

- Connect application with MongoDB.
 - Configure application port.
 - Configure database name.
-

9. REST APIs Implemented

Student APIs

Method	Endpoint
--------	----------

POST	/students
------	-----------

GET	/students
-----	-----------

GET	/students/{id}
-----	----------------

PUT	/students/{id}
-----	----------------

DELETE	/students/{id}
--------	----------------

Hostel APIs

Method	Endpoint
--------	----------

POST	/hostels
------	----------

GET	/hostels
-----	----------

PUT	/hostels/{id}
-----	---------------

DELETE	/hostels/{id}
--------	---------------

Room APIs

Method	Endpoint
--------	----------

POST	/rooms
------	--------

GET	/rooms
-----	--------

PUT	/rooms/{id}
-----	-------------

DELETE	/rooms/{id}
--------	-------------

Staff APIs

Method Endpoint

POST /staff

GET /staff

PUT /staff/{id}

DELETE /staff/{id}

Fee APIs

Method Endpoint

POST /fees

GET /fees

PUT /fees/{id}

DELETE /fees/{id}

Department APIs

Method Endpoint

POST /departments

GET /departments

PUT /departments/{id}

DELETE /departments/{id}

Visitor APIs

Method Endpoint

POST /visitors

Method Endpoint

GET /visitors

PUT /visitors/{id}

DELETE /visitors/{id}

Complaint APIs

Method Endpoint

POST /complaints

GET /complaints

PUT /complaints/{id}

DELETE /complaints/{id}

10. Postman Testing

For report validation, include screenshots of successful API execution.

Screenshot 3

Add Student (POST)

POST http://localhost:8080/students

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "name": "Yash Sharma",
3   "email": "yash@gmail.com",
4   "phone": "9876543211",
5   "departmentId": "D01",
6   "roomId": "R101"
7 }
```

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "departmentId": "D01",
3   "email": "yash@gmail.com",
4   "id": "6a2ffc1c1bce1eb939275ee0",
5   "name": "Yash Sharma",
6   "phone": "9876543211",
7   "roomId": "R101"
8 }
```

Explanation

A new student record is created and stored in MongoDB.

Screenshot 4

Get All Students (GET)

GET http://localhost:8080/students

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "name": "Yash Sharma",
3   "email": "yash@gmail.com",
4   "phone": "9876543211",
5   "departmentId": "PASTE_DEPARTMENT_ID_HERE",
6   "roomId": "PASTE_ROOM_ID_HERE"
7 }
```

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

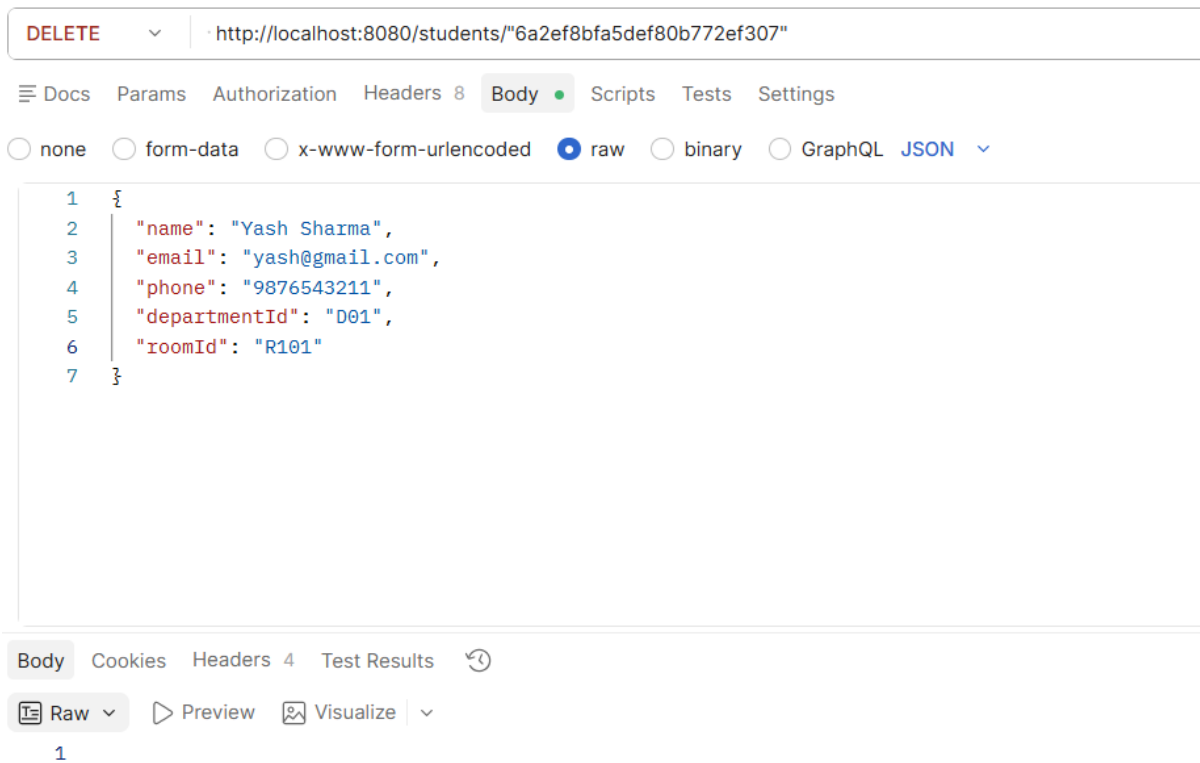
```
1 [
2   {
3     "departmentId": "D001",
4     "email": "yash@gmail.com",
5     "id": "6a2d987d9811906dfb502d64",
6     "name": "Yash",
7     "phone": "9876543210",
8     "roomId": "R101"
9   },
10  {
11    "departmentId": "D001",
12    "email": "yash@gmail.com",
13    "id": "6a2d9b749811906dfb502d68",
14    "name": "Yash",
15    "phone": "9876543210",
16    "roomId": "R101"
17  },
18  {
19    "departmentId": "D001",
20    "email": "yash@gmail.com",
21    "id": "6a2ef8bfa5def80b772ef307",
22    "name": "Yash ",
23    "phone": "9876543211"
```

Explanation

Retrieves all student records from the database

Screenshot 5

Delete Student (DELETE)



Explanation

Deletes student record successfully.

Screenshot 6

Add Hostel

POST http://localhost:8080/hostels

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "hostelName": "Boys Hostel A",
3   "hostelType": "Boys"
4 }
```

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "hostelName": "Boys Hostel A",
3   "hostelType": "Boys",
4   "id": "6a2ff9eb1bce1eb939275edd"
5 }
```

Screenshot 7

Add Room

HTTP **http://localhost:8080/rooms**

POST **http://localhost:8080/rooms**

Docs Params Authorization Headers 8 **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "roomNumber": "101",
3   "capacity": 4,
4   "occupiedSeats": 0,
5   "hostelId": "6a2ff9eb1bce1eb939275edd"
6 }
```

Body Cookies Headers 5 Test Results ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "capacity": 4,
3   "hostelId": "6a2ff9eb1bce1eb939275edd",
4   "id": "6a2ffb191bce1eb939275ede",
5   "roomNumber": "101"
6 }
```

Screenshot 8

Add Staff

POST http://localhost:8080/staff

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "name": "Rajesh Kumar",
3   "designation": "Warden",
4   "phone": "9876543210",
5   "hostelId": "PASTE_HOSTEL_ID_HERE"
6 }
```

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "designation": "Warden",
3   "hostelId": "PASTE_HOSTEL_ID_HERE",
4   "id": "6a3149789977a5ceb90f3bbe",
5   "name": "Rajesh Kumar",
6   "phone": "9876543210"
7 }
```

Screenshot 9

Add Fee Record

POST

http://localhost:8080/fees

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

"studentId": "6a2ff9aa1bce1eb939275edc" ,

3

"amount": 5000,

4

"month": "June 2026"

5

}

Body

Cookies

Headers 5

Test Results 1/1

{}

JSON

Preview

Visualize

1

{

2

"amount": 5000.0,

3

"id": "6a3164289977a5ceb90f3bc3",

4

"month": "June 2026",

5

"status": "Pending",

6

"studentId": "6a2ff9aa1bce1eb939275edc"

7

}

Screenshot 10

Add Visitor

POST

http://localhost:8080/visitors

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

```
1 {
2   "visitorName": "Ramesh Sharma",
3   "phone": "9812345678",
4   "studentId": "6a314a529977a5ceb90f3bbf",
5   "purpose": "Family Visit"
6 }
```

Body

Cookies

Headers 5

Test Results

{}

JSON

Preview

Visualize

```
1 {
2   "id": "6a314de79977a5ceb90f3bc0",
3   "phone": "9812345678",
4   "purpose": "Family Visit",
5   "studentId": "6a314a529977a5ceb90f3bbf",
6   "visitDate": "2026-06-16T18:51:43.0889348",
7   "visitorName": "Ramesh Sharma"
8 }
```

Screenshot 11

Add Complaint

POST

http://localhost:8080/complaints

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

"studentId": "6a2ff9aa1bce1eb939275edc",

3

"description": "Water supply issue in room 101"

4

}

Body

Cookies

Headers 5

Test Results 1/1

{}

JSON

Preview

Visualize

1

{

2

"assignedStaffId": null,

3

"description": "Water supply issue in room 101",

4

"id": "6a3166849977a5ceb90f3bc4",

5

"status": "Pending",

6

"studentId": "6a2ff9aa1bce1eb939275edc"

7

}

11. Output

Expected Results

- Student records successfully managed.

test

students

+

localhost:27017 > test > students

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

▶

```
_id: ObjectId('6a2d987d9811906dfb502d64')
name: "Yash"
email: "yash@gmail.com"
phone: "9876543210"
departmentId: "D001"
roomId: "R101"
_class: "com.example.demo.model.Student"
```

```
_id: ObjectId('6a2d9b749811906dfb502d68')
name: "Yash"
email: "yash@gmail.com"
phone: "9876543210"
departmentId: "D001"
roomId: "R101"
_class: "com.example.demo.model.Student"
```

```
_id: ObjectId('6a2ef8bfa5def80b772ef307')
name: "Yash "
email: "yash@gmail.com"
phone: "9876543211"
departmentId: "D001"
roomId: "R101"
_class: "com.example.demo.model.Student"
```

- Hostel information maintained.

test

hostels

+

localhost:27017 > test > hostels

Documents 3

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

+ ADD DATA

BULK

EXPORT DATA

EXPORT CODE

```
_id: ObjectId('6a2e5d3c795dd77d73a1867d')
hostelName: "Boys Hostel"
hostelType: "Boys"
_class: "com.example.demo.model.Hostel"
```

```
_id: ObjectId('6a2ef460a5def80b772ef304')
hostelName: "Boys Hostel A"
hostelType: "Boys"
_class: "com.example.demo.model.Hostel"
```

```
_id: ObjectId('6a2ff9eb1bce1eb939275edd')
hostelName: "Boys Hostel A"
hostelType: "Boys"
_class: "com.example.demo.model.Hostel"
```

- Room allocations stored.

test

rooms

+

localhost:27017 > test > rooms

Documents 3

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

```
_id: ObjectId('6a2efbe05c409647bd3781e0')
roomNumber: "101"
capacity: 4
hostelId: "6a2ef8bfa5def80b772ef307"
_class: "com.example.demo.model.Room"
```

```
_id: ObjectId('6a2f89f066bb0b00f5489c29')
roomNumber: "101"
capacity: 4
hostelId: "6a2ef8bfa5def80b772ef307"
_class: "com.example.demo.model.Room"
```

```
_id: ObjectId('6a2ffb191bce1eb939275ede')
roomNumber: "101"
capacity: 4
hostelId: "6a2ff9eb1bce1eb939275edd"
_class: "com.example.demo.model.Room"
```

- Staff information tracked.

test

staff

+

localhost:27017 > test > staff

Documents 5

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

▶

```
_id: ObjectId('6a2e47fecfcf305326cfbd81')
name: "Amit"
designation: "Warden"
phone: "9999999999"
hostelId: "H001"
_class: "com.example.demo.model.Staff"
```

```
_id: ObjectId('6a2ef712a5def80b772ef306')
name: "Rajesh Kumar"
designation: "Warden"
phone: "9876543210"
hostelId: "6a2ef460a5def80b772ef304"
_class: "com.example.demo.model.Staff"
```

```
_id: ObjectId('6a2ffb8f1bce1eb939275edf')
name: "Rajesh Kumar"
designation: "Warden"
phone: "9876543210"
hostelId: "6a2ff9eb1bce1eb939275edd"
_class: "com.example.demo.model.Staff"
```

- Fee records managed.

test fees +

localhost:27017 > test > fees

Documents 2 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ↗

+ ADD DATA BULK EXPORT DATA EXPORT CODE

```
_id: ObjectId('6a2ef178a5def80b772ef302')
studentId : "6a2ef0c0a5def80b772ef301"
amount : 5000
status : "Pending"
month : "June 2026"
_class : "com.example.demo.model.Fee"
```

```
_id: ObjectId('6a3164289977a5ceb90f3bc3')
studentId : "6a2ff9aa1bce1eb939275edc"
amount : 5000
status : "Pending"
month : "June 2026"
_class : "com.example.demo.model.Fee"
```

- Visitor entries recorded.

test

visitors

+

localhost:27017 > test > visitors

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT DATA

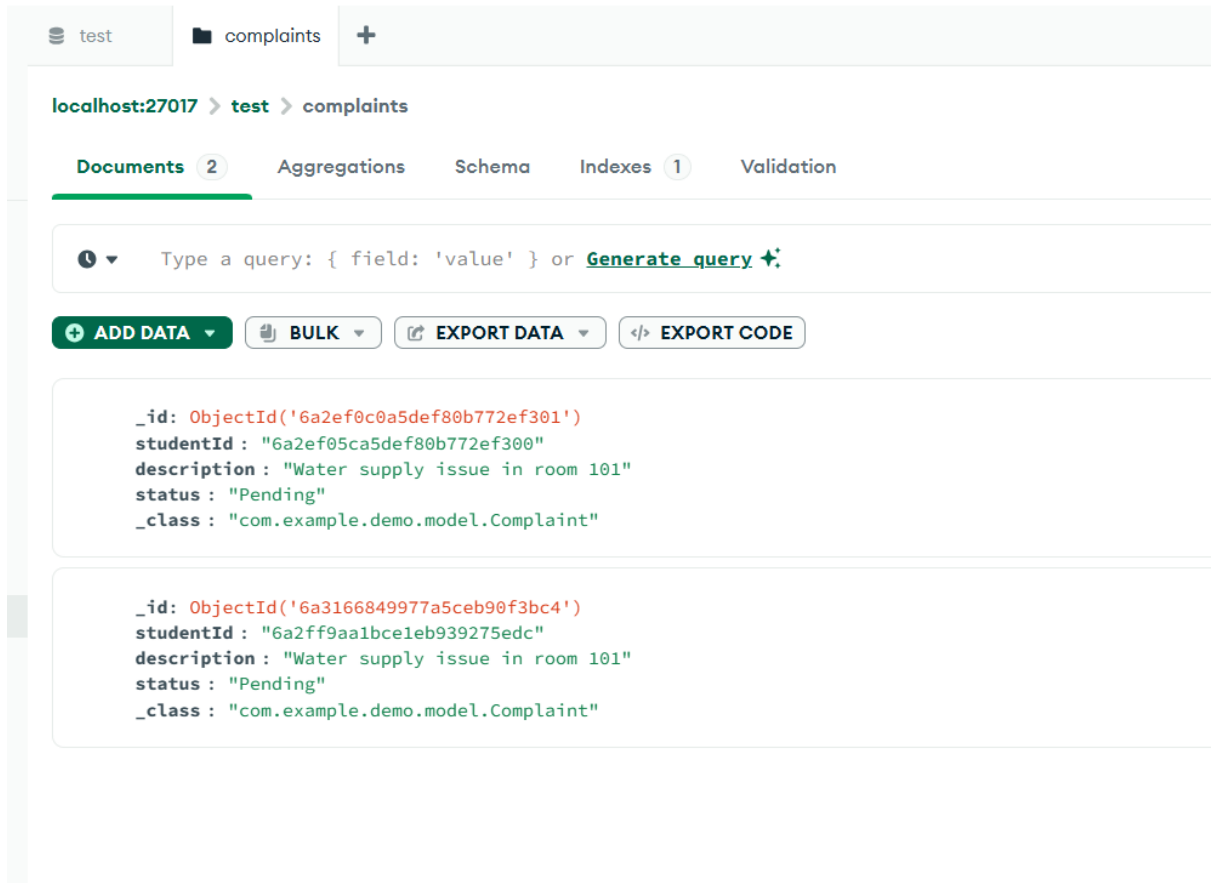
EXPORT CODE

```
_id: ObjectId('6a2ef05ca5def80b772ef300')
visitorName: "Ramesh Sharma"
phone: "9812345678"
studentId: "6a2e5eea31f5016b99306b6a"
visitDate: 2026-06-14T18:18:04.263+00:00
purpose: "Family Visit"
_class: "com.example.demo.model.Visitor"
```

```
_id: ObjectId('6a314de79977a5ceb90f3bc0')
visitorName: "Ramesh Sharma"
phone: "9812345678"
studentId: "6a314a529977a5ceb90f3bbf"
visitDate: 2026-06-16T13:21:43.088+00:00
purpose: "Family Visit"
_class: "com.example.demo.model.Visitor"
```

```
_id: ObjectId('6a315f0f9977a5ceb90f3bc1')
studentId: "6a2ff9aa1bce1eb939275sdc"
visitDate: 2026-06-16T14:34:55.022+00:00
_class: "com.example.demo.model.Visitor"
```

- Complaints tracked and resolved.



12. Outcome

Successfully developed a complete backend-based Hostel Management System using Spring Boot and MongoDB that provides RESTful APIs for managing hostel operations. The application follows a layered architecture and supports CRUD operations for all major hostel entities.

13. Conclusion

The Hostel Management System simplifies hostel administration by digitizing student records, room management, staff information, fee collection, visitor tracking, department management, and complaint handling. Spring Boot provides a scalable backend framework while MongoDB offers flexible and efficient data storage. The system is fully testable using Postman and documented through Swagger, making it suitable for academic projects as well as future real-world extensions.
